

Handwritten Digit Recognition with a Feedforward Neural Network

Classifying the MNIST dataset in Keras and comparing two hidden-layer sizes

SUBMITTED BY
Kavita P

UNIVERSITY
Rai Technology University

USN
RTU24101CS050

01 Objective

This assignment builds, trains and evaluates an artificial neural network that recognises handwritten digits (0–9) from the **MNIST** dataset. A baseline network with 128 hidden neurons (**Model A**) is compared against a wider network with 256 hidden neurons (**Model B**) to see how hidden-layer size affects accuracy. Everything is implemented in Python with **TensorFlow/Keras**, and **Matplotlib** is used for the visualisations.

02 Dataset and Pre-processing

MNIST was loaded directly with `keras.datasets.mnist`. Each image is a single-channel 28×28 grid of pixel intensities from 0 (black) to 255 (white).

60,000

Training images

10,000

Test images

28×28

Greyscale pixels per image

10

Classes (digits 0–9)

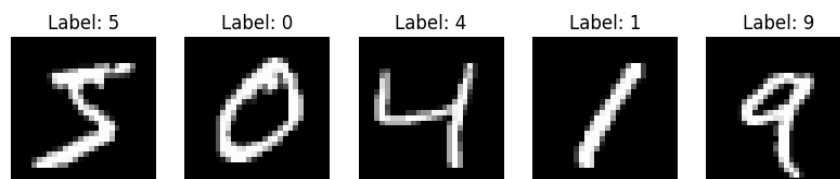


Figure 1. First five training images with their labels (5, 0, 4, 1, 9).

Pre-processing. Pixel values were divided by 255 so that every input lies between 0 and 1; scaled inputs keep gradients well-behaved and help the optimiser converge faster. During training, 10% of the training set (6,000 images) was held out for validation, leaving 54,000 images for weight updates. The 10,000 test images were used only for the final evaluation.

03 Model Architecture

Both models are fully connected networks built with `keras.Sequential`. The image is flattened into a 784-value vector, passed through one hidden dense layer with ReLU activation, and mapped to ten softmax outputs — one probability per digit. Only the width of the hidden layer differs between the two models.



Figure 2. Network architecture (hidden layer: 128 neurons in Model A, 256 in Model B).

Configuration	Model A (baseline)	Model B (modified)
Hidden layer	Dense(128, ReLU)	Dense(256, ReLU)
Output layer	Dense(10, Softmax)	Dense(10, Softmax)
Trainable parameters	101,770	203,530
Optimiser	Adam	Adam
Loss function	Sparse categorical cross-entropy	Sparse categorical cross-entropy
Epochs / validation split	5 / 10%	5 / 10%

Design choices

- **ReLU** in the hidden layer is cheap to compute and avoids the vanishing-gradient problem of sigmoid/tanh.
- **Softmax** turns the ten outputs into probabilities that sum to 1; the largest one is the predicted digit.
- **Adam** adapts the learning rate of each weight, so it trains reliably with default settings.
- **Sparse categorical cross-entropy** suits multi-class problems with integer labels, so no one-hot encoding is needed.

04 Results

Each model was trained for 5 epochs with the Keras default batch size of 32 (1,688 steps per epoch). Training accuracy climbed above 98.5% for both models, while validation accuracy settled between 97.7% and 98.1%. On the unseen test set:

97.71%

Model A test accuracy (loss 0.0769)

97.80%

Model B test accuracy (loss 0.0714)

+0.09 pp

Gain from doubling the hidden layer — 9 more images correct

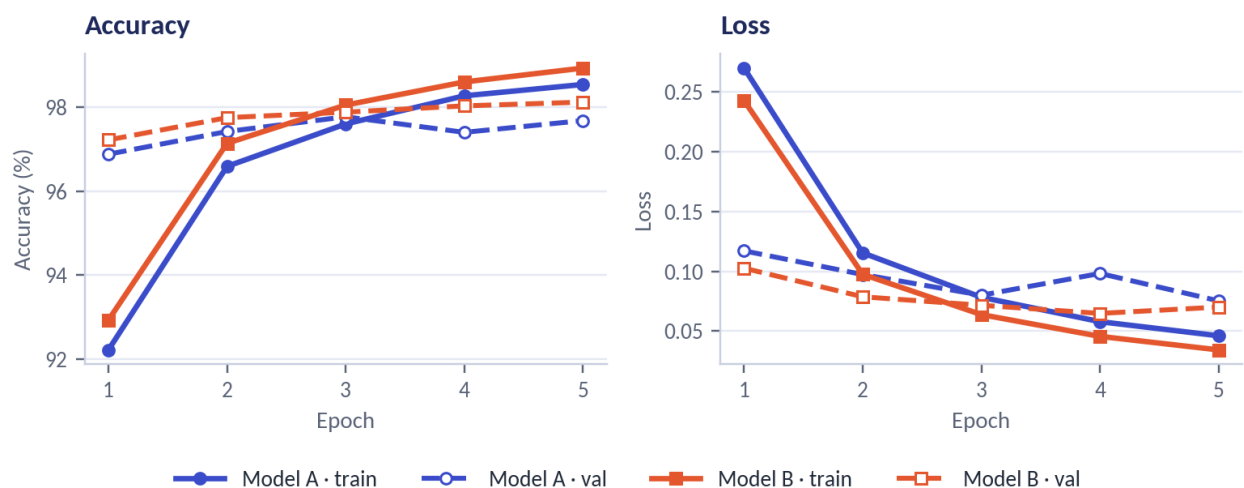


Figure 3. Training (solid) and validation (dashed) accuracy and loss for both models over five epochs.

As a sanity check, Model A was asked to classify the first five test images. All five predictions matched the true labels (Figure 4).

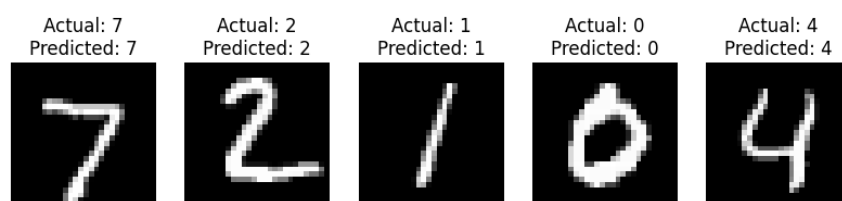


Figure 4. Model A predictions on the first five test images (actual vs. predicted).

05 Comparison and Discussion

Metric	Model A (128)	Model B (256)
Final training accuracy	98.54%	98.93%
Final validation accuracy	97.68%	98.12%
Final training loss	0.0461	0.0343
Final validation loss	0.0753	0.0701
Test accuracy	97.71%	97.80%
Test loss	0.0769	0.0714
Approx. time per epoch	6–7 s	9–13 s

- **Both models learn fast.** Training accuracy is above 92% after the first epoch and validation accuracy is already near 97%, showing that MNIST is well suited to even a single-hidden-layer network.
- **The wider network fits better.** Model B reached a lower training loss and a higher validation accuracy at every epoch, since extra neurons give it more capacity to learn useful features.
- **The test-set gain is small.** A 0.09-point improvement is 9 images out of 10,000. With one run per model and random weight initialisation, a difference this size may be within run-to-run variation, so Model B is best described as “slightly better”, not conclusively better.
- **Early signs of overfitting.** Validation loss rose in epoch 4 for Model A (0.0800 → 0.0982) and in epoch 5 for Model B (0.0648 → 0.0701) while training loss kept falling, and in both models training accuracy now sits roughly 0.8–0.9 points above validation accuracy.
- **Cost.** Model B has twice the parameters and takes roughly 1.5× longer per epoch.

06 Conclusion

A simple feedforward network trained for only five epochs classifies MNIST digits with about 98% test accuracy. Increasing the hidden layer from 128 to 256 neurons improved test accuracy from 97.71% to 97.80% and lowered the loss, at the price of double the parameters and longer training. The gain is modest, which suggests that widening a fully connected layer has diminishing returns on this task.

Possible improvements: add Dropout or early stopping to control overfitting, repeat each experiment with several random seeds to confirm the difference, inspect a confusion matrix to see which digits are confused, and try a convolutional neural network, which typically exceeds 99% on MNIST.

KEY TAKEAWAY

A small dense network reaches **~98% accuracy** on MNIST in five epochs; doubling the hidden layer adds only **+0.09 points** while doubling the parameter count.